

Daily Scholar Papers Report — 2026-05-25

2026-05-25

Daily Scholar Papers Report — 2026-05-25

[Download PDF](#)

Window covered: 2026-05-24 → 2026-05-25 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Today's window surfaced four Scholar-alert threads carrying thirteen distinct candidates plus two user-curated self-emails; nine cleared Stage-1 triage (three on followed-researcher signal, four on the preference-aware preprint posterior of 0.75, and two as user-picks that bypass saturation/topicality filters). The four Outstanding deep-reads converge on a single methodological move — **bound the LLM to semantic judgment and discharge every soundness-relevant verdict on a deterministic backend (BMC solver, debugger oracle, graph-reachability query, or symbolic-execution trace): “Agentic Model Checking”** (Sun, Liu, Kroening, Xue — MBZUAI / Amazon) couples LLM agents with CBMC/Kani under “agents propose, solvers verify,” confirming 34 realistic bugs in a 15 kLoC LLM-generated ARM64 kernel, 25 in a 50 kLoC LLM-generated Rust C compiler, and an MMIO bounds-check bypass in an out-of-tree Realtek r8125 driver; **“Teaching LLMs Program Semantics via Symbolic Execution Traces”** (Bayer, Zetsche, Bouissou, Delmas, Tautschnig, Kong — Cambridge / AWS) shows that continued pretraining of Qwen3-8B on ~3,000 Soteria bug traces plus chain-of-thought yields a *superadditive* +17.9 pp on detecting C-property violations (neither alone helps), letting the 8B model beat 4× larger Qwen3-32B on violation detection; **“No Attack Required: Semantic Fuzzing for Specification Violations in Agent Skills”** (Li, Wen, Chen, Liu, Tian, Feng — UCLA / UCSB / UCSD) reduces NL guardrails to reachability goals $\phi = (\pi s, \pi d, \Pi g)$ over annotated execution traces, finds violations in 120 of 402 deployed agent skills (29.9 %, including 26 zero-day exploitable violations) in ~11 min/skill with Thompson-Sampling-guided semantic mutation; **“Veritas”** (Zheng, Pesoli, Valleri, Jana, Cavallaro — UCL / Bynario / Columbia) grounds binary memory-corruption detection in a static slicer over RetDec-lifted LLVM IR plus a debugger-based validator, reaching 90 % recall on real-world binaries, zero false positives across 623 exhaustively-verified candidates, and disclosing a new Apple CVE. Four Keep papers extend orthogonal axes — **SoK: Modularized Symbolic Execution** (Mattei et al., SecDev '26) systematises 225 SE papers into five canonical modules; **iDetector** (TOSEM 2026) tackles modern Spring-era Java web injection paths; **UNTRUSTVUL** (TSE 2026) automates post-hoc identification of untrustworthy alerts from ML vulnerability detectors; **Beyond the Tip of the Iceberg** (Minn et al., David Lo group) shows that 27 %/40 % of Dockerfile SATD admission/repayment events couple to non-Dockerfile artefacts. The Borderline paper is followed-researcher-promoted off-topic robotics.

Outstanding: 4 · **Keep:** 4 · **Borderline High-Priority:** 1

The full analysis follows.

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Agentic Model Checking	Youcheng Sun, Jiawen Liu, Daniel Kroening, Jason Xue	arXiv preprint, 2026	arXiv:2605.21434
1.2	Teaching LLMs Program Semantics via Symbolic Execution Traces	Jonas Bayer, Stefan Zetsche, Olivier Bouissou, Remi Delmas, Michael Tautschnig, Soonho Kong	arXiv preprint, 2026	arXiv:2605.06184
1.3	No Attack Required: Semantic Fuzzing for Specification Violations in Agent Skills	Ying Li, Hongbo Wen, Yanju Chen, Hanzhi Liu, Yuan Tian, Yu Feng	arXiv preprint, 2026	arXiv:2605.13044
1.4	Veritas: A Semantically Grounded Agentic Framework for Memory Corruption Vulnerability Detection in Binaries	Xinran Zheng, Alfredo Pesoli, Marco Valleri, Suman Jana, Lorenzo Cavallaro	arXiv preprint, 2026	arXiv:2605.15097
2.1	SoK: A Modularized Framework for Symbolic Execution and Application for Usable Tool Design	James Mattei, Andrew Lin, Jasper Geer, Jie Hu, Moritz Schloegel, Tiffany Bao, Daniel Votipka	ACM SecDev '26 (CC-BY 4.0)	DOI 10.1145/3805773.3805996
2.2	iDetector: Unraveling and Automating the Detection of Modern Java Web Injection Vulnerabilities	Xiang Chen, Yu Li, Zhuo Jin, Yong Tan, Xiang Li, Bo Wang, Min Zhang et al.	ACM TOSEM, 2026	DOI 10.1145/3808700
2.3	UNTRUSTVUL: Automated Untrustworthy Alert Identification in Vulnerability Detection Models	Lam Nguyen Tung, Xiaoning Du, Neelofar Neelofar, Aldeida Aleti	IEEE TSE, 2026	IEEE Xplore 11522575
2.4	Beyond the Tip of the Iceberg: Understanding SATD in Dockerfiles through the Lens of Co-evolution	Wei Minn, Yan Naing Tun, Biniam Fesseha Demissie, Rui'ang Hu, Jiakun Liu, Mariano Ceccato, Lwin Khin Shar, David Lo	arXiv preprint, 2026	arXiv:2605.21238

Outstanding Deep-Reads

1.1 · AGENTIC-BMC · An “agents propose, solvers verify” paradigm pairing LLM agents with CBMC/Kani — confirms 34 realistic bugs in a 15 kLoC LLM-generated ARM64 kernel and 25 in a 50 kLoC LLM-generated Rust C compiler, with a four-stage realism-audit pipeline that classifies counterexamples instead of reporting them

1.1 Agentic Model Checking

[arXiv:2605.21434](https://arxiv.org/abs/2605.21434) · [Project: AProver / BMC-Agent](#)

Title: Agentic Model Checking **Authors:** Youcheng Sun, Jiawen Liu, Daniel Kroening, Jason Xue
Affiliations: Mohamed bin Zayed University of Artificial Intelligence (UAE) · Amazon (USA) **Venue:** arXiv preprint, 2026-05-20. **License:** arXiv non-exclusive distribution. **Source signal:** Scholar alert “Daniel Kroening - new articles” 2026-05-24, plus preference-promoted preprint (venue::preprint posterior 0.75).

Thesis

Verifying LLM-generated systems code is hard for three compounding reasons: bugs are prevalent, formal specifications are missing, and safety contracts are encoded implicitly at call sites rather than enforced at function boundaries. Existing LLM+-verification work splits into two unattractive camps — LLM-only pipelines that scale and produce readable explanations but are unsound for arithmetic and memory safety, and LLM-plus-deductive-checker pipelines that are sound but cannot produce concrete counterexamples and stall on loop-heavy code. **Agentic model checking** is a third design point: agents perform every task that requires *semantic judgment* (spec inference, check selection, counterexample classification, refinement proposal), while a **bounded model checking** backend (CBMC for C, Kani for Rust) discharges every *soundness-relevant decision*. The slogan: *agents propose, solvers verify*. Every LLM output that could change a verification verdict is forced through a deterministic check — a DSL parser, a BMC reachability query, an SMT-based soundness guard, or concrete execution.

Architecture

Each function is checked in isolation against its inferred spec, with callees replaced by postcondition-constrained stubs so per-query cost scales with one function’s state space, and accepted refinements propagate to every caller’s harness through a shared spec store. Stage-by-stage:

- 1. Top-down specification generation.** An LLM agent traverses the call graph from no-caller roots toward leaves, retrieving each function body plus its caller context and a compact codebase-wide domain summary, then drafting pre/postcondition pairs in a restricted DSL. The DSL is engineered to translate deterministically into `__CPROVER_assume / __CPROVER_assert` on the C path and `kani::assume / kani::assert` on the Rust path; per-backend adapters handle predicates without a direct equivalent on the other side. Mutually recursive functions are condensed into SCC nodes via Kosaraju’s algorithm and specified as a unit. Optional functional-correctness clauses (reference-equivalence, algebraic identities, fold expressions, round-trip properties) lift verification from panic-freeness to behavioural faithfulness and are AND-merged into the postcondition so one BMC query discharges both obligations.
- 2. Per-function flag selection.** A second LLM agent picks which optional backend checks are semantically warranted for each function — on the C path among `--unsigned-overflow-check`, `--signed-overflow-check`, `--conversion-check`, `--pointer-overflow-check`,

with Kani analogues on the Rust path. Selector runs in parallel across functions and defaults conservatively (all flags off) on any LLM failure.

3. **Per-function BMC.** Default unwinding bound $k = 4$, per-function timeout 120 s. Retry chains absorb transient inconclusive results: CBMC `too many addressed objects` retries with larger `--object-bits`; Kani unwind-exhausted retries with the bound quadrupled ($4 \rightarrow 16$); Kani timeouts retry with slice bounds shrunk $4 \rightarrow 2 \rightarrow 1$.
4. **Counterexample deduplication** by `(function, property type)` representative classes.
5. **Four-stage validation pipeline.** *Input reachability* propagates the witness inputs up the call graph and records where propagation halts (no-caller boundary vs. mid-chain), setting the final evidence tier (*confirmed system entry* vs *confirmed bmc*). *Callee feasibility* substitutes real callee bodies for stubs, pins the witness, and re-runs BMC; survivors are symbolically reachable from a real call site. *Dynamic replay* compiles a GCC harness and executes the witness, capturing SIGSEGV/SIGABRT/SIGFPE/SIGILL to upgrade the finding to *confirmed dynamic*. *Realism audit* runs first a pattern-detector layer (library-init globals set to NULL, USB-serial framework-managed pointers, NULL-guarded deref contradictions, path-divergent unwinds), then an LLM judge that reads spec / harness / witness / surrounding source; uncertain verdicts fold into *unrealistic* so any non-realistic finding is treated uniformly.
6. **Adaptive refinement loop.** Unrealistic / uncertain witnesses trigger an LLM that proposes verifier-level / caller-side / callee-spec refinements; a soundness guard checks they don't mask real bugs; accepted refinements persist in the verification knowledge base and the witness-pattern library, reused across runs.

Headline results

The contribution list (§1) reports four evaluation corpora spanning both supported languages:

Corpus	Size / nature	Result claimed
VibeOS	15,000 LoC LLM-generated ARM64 kernel, C	34 realistic bugs across 12 modules; 16 reproduced as runtime faults
Five mature OSS targets (jq, OpenSSL, libcurl, libxml2, protobuf upb)	C, heavily OSS-Fuzz-hardened	2 undefined-behaviour defects disclosed in jq; bounded clean verifications on hardened parser surfaces
Realtek r8125 Linux driver	Out-of-tree kernel module, C	<code>CAP_NET_ADMIN</code> -gated MMIO bounds-check bypass in <code>rtl8125_tool_ioctl</code>
<code>claudes-c-compiler</code>	50,000-line LLM-generated Rust C compiler	25 real bugs: 24 panic-class defects on public-API byte helpers + 1 functional-correctness violation caught only by behavioural specs; bounded functional equivalence for ELF hash and header-write helpers

Why it matters

The walkthrough example in §2 illustrates the architectural payoff. A `kapi_file_size` function in VibeOS guards a `void*` against NULL but then casts it to `vfs_node_t*` without further validation. A reviewer

sees a defensive function; the LLM-generated postcondition — *if node is null return -1, else the returned value must equal (int) node->size* — makes the missing obligation explicit. CBMC finds a non-null pointer that violates the postcondition; classification confirms `kapi_file_size` is exposed via the `kapi` dispatch table to user-space callers; dynamic replay captures a SIGSEGV on a non-null unallocated pointer; the realism audit confirms the witness corresponds to inputs an application can supply. No single stage suffices: the LLM cannot prove the postcondition across all inputs, BMC alone cannot confirm reachability, and dynamic execution alone cannot find the witness in the first place. *The bug is what the null check doesn't protect against.* The same shape — LLM for the semantic judgment that opens the door, deterministic backend for the discharge — explains why this paper sets the methodological frame for the next year of LLM-in-verification work.

1.2 · LLM-TRACE-PRETRAIN · [USER-PICK] An evaluation framework of 500 SV-COMP C verification tasks + an annotation-free training recipe: ~3,000 Soteria symbolic-execution bug traces continue-pretrained into Qwen3-8B produce a superadditive +17.9 pp on violation detection (CoT alone ≈ no gain, traces alone ≈ no gain) and beat the 4× larger Qwen3-32B

1.2 Teaching LLMs Program Semantics via Symbolic Execution Traces

[arXiv:2605.06184](https://arxiv.org/abs/2605.06184)

Title: Teaching LLMs Program Semantics via Symbolic Execution Traces **Authors:** Jonas Bayer, Stefan Zetzsche (corresponding), Olivier Bouissou, Remi Delmas, Michael Tautschnig, Soonho Kong
Affiliations: University of Cambridge · Amazon Web Services (work done during Bayer's internship at AWS) **Venue:** arXiv preprint, 2026-05-07. **License:** arXiv non-exclusive distribution; SV-COMP 2025 benchmark infrastructure under Apache 2.0; CodeParrot subject to per-repository licenses. **Source signal:** User self-email 2026-05-24 11:17:55 UTC ("Paper"); also preference-promoted preprint.

Thesis

For higher-level languages, code-understanding evaluations are approaching saturation. C requires reasoning about memory safety, pointer arithmetic, and undefined behaviour — three places models remain less reliable — yet there is no established framework that isolates *this kind* of semantic reasoning for C. The paper builds one on top of SV-COMP 2025 (the Software Verification Competition): 500 verification tasks across five property types (memory safety, overflow, termination, reachability, data races), with ground truth from formal-verification tools. Baseline evaluation of 14 models across six families exposes a sharp, previously-masked weakness: while most models exceed 90 % accuracy on confirming that properties hold, **four of fourteen correctly identify fewer than 50 % of actual violations**, and violation accuracy drops sharply with program length — one model falls below 10 % on 100–200-LoC programs. Model size and code-specialisation do not straightforwardly predict performance; GPT-OSS-20B detects violations more reliably than Mistral Large 3 675B and Qwen3-Coder-480B, suggesting training data and objectives matter more than scale.

Training pipeline

The fix is to train on formal-verification artifacts:

1. Filter the CodeParrot GitHub dataset to 1 M self-contained C files.

- 2. Run the open-source **Soteria** symbolic-execution engine on each file; ~34,000 usable traces emerge (yield bounded by Soteria timeout/parser limits — expected to improve as the tooling matures).
- 3. Filter to the **manifest-bug subset** (cases where Soteria proves a violation occurs on all feasible inputs): 3,208 traces.
- 4. Continue-pretrain Qwen3-8B on those traces for ~10 minutes of training time.
- 5. At inference time, enable chain-of-thought reasoning.

Headline numbers

Setting	Effect on Qwen3-8B
Manifest-bug traces + CoT (combined)	+17.9 pp on detecting property violations
Holds accuracy (reported overall)	90.7 % → 78.0 % (drop driven <i>entirely</i> by token-budget exhaustion)
Holds accuracy on parseable responses only	90.8 % vs 91.2 % baseline (preserved)
Violated accuracy on parseable responses	+26 pp
Trace pretraining alone	≈ no meaningful gain
CoT alone	≈ no meaningful gain
Comparison to Qwen3-32B (no thinking)	trained 8B wins on violation detection , approaches it on overall accuracy

28 configurations confirm gains stem from trace semantics, not code volume (training on the underlying C files alone also hurts), and that trace curation and format matter. Improvements transfer across all five property types — including the four that the training traces do not directly target.

Why it matters

This is an *annotation-free* recipe: any sound symbolic-execution engine with a trace output (KLEE, SymCC, Manticore, Soteria) is a candidate training-signal source, and the per-trace cost is dominated by running the analyser rather than human labelling. The **superadditivity** of trace pretraining × CoT is the real result — neither alone produces meaningful gains, but their combination does. The reading is that trace pretraining teaches the model *how to use a trace as a thinking scaffold*, which only pays off when inference is also given thinking time. The negative ablations are the persuasive part; the headline number is the consequence.

1.3 · AGENT-SKILL-FUZZ · [USER-PICK] Sefz reduces natural-language agent-skill guardrails to reachability goals $\phi = (\pi_s, \pi_d, \Pi_g)$ over annotated execution traces and Thompson-Sampling-guided semantic mutation — 120/402 deployed skills violated (29.9 %), 26 zero-day exploitable violations, average 11 min/skill, with a deterministic graph oracle instead of an LLM judge

1.3 No Attack Required: Semantic Fuzzing for Specification Violations in Agent Skills

[arXiv:2605.13044](#)

Title: No Attack Required: Semantic Fuzzing for Specification Violations in Agent Skills **Authors:** Ying Li, Hongbo Wen, Yanju Chen, Hanzhi Liu, Yuan Tian, Yu Feng **Affiliations:** University of California, Los

Thesis

LLM-powered agents act on user requests by invoking skills — bundles of natural-language instructions plus optional executable scripts that an agent discovers, loads, and follows. Skill specifications declare **guardrails** in prose (“require explicit confirmation before deletion”, “the `--confirm-publish` flag must be set”), but these can fail silently on benign inputs because the guardrail’s semantics are undefined for autonomous execution, or because the implementation silently ignores the constraint, or because the composition of individually safe actions violates an unwritten invariant. The paper names this failure mode **specification violations** — invisible to static analysers, traditional fuzzers, and prompt-injection defences alike — and presents **Sefz**, a goal-directed semantic fuzzer that surfaces them automatically.

Definitions (paper §IV)

The formalism is unusually clean.

An **annotated execution trace** is a finite event sequence $\tau = \langle e_1, \dots, e_n \rangle$ with typed dependency edges, forming a labelled DAG. Events are one of four types — `skill` (top-level invocation), `tool` (concrete tool call), `resource` (terminal data source/sink), `auth` (confirmation prompt / permission check) — and edges are one of three — `invoke` (caller–callee), `dataflow` (output feeds another’s arguments), `control` (sequential ordering). Each event carries a predicate label $\lambda(e) \subseteq P$ from a five-predicate vocabulary:

Predicate	Meaning
<code>tainted</code>	event handles externally-controlled data
<code>sens</code>	event accesses or produces sensitive data
<code>ask</code>	event is a user-confirmation checkpoint
<code>exec</code>	event modifies state
<code>cred</code>	event reads or transmits authentication material

A **reachability goal** is a triple $\phi = (\pi_s, \pi_d, \Pi_g) : \text{source set, sink set, gate set}$. The satisfaction relation $\tau \models \phi$ holds iff some dependency chain $e_i \rightarrow \dots \rightarrow e_j$ has $\lambda(e_i) \supseteq \pi_s$, $\lambda(e_j) \supseteq \pi_d$, and $\lambda(e_m) \cap \Pi_g = \emptyset$ for every intermediate e_m — i.e. source-to-sink with no intervening gate event. Three templates instantiate the framework:

- **Unconfirmed action** (Eq. 1 in the paper): $\phi_u = (\{\text{tainted}\}, \{\text{exec}\}, \{\text{ask}\})$.
- **Data exfiltration**: $\phi_e = (\{\text{sens}\}, \{\text{exec}, \text{tainted}\}, \{\text{ask}\})$.
- **Privilege escalation**: $\phi_p = (\{\text{tainted}\}, \{\text{cred}\}, \{\text{ask}\})$.

Sefz refuses to let an LLM translate a guardrail directly into a goal (the goal is the ground truth that the whole pipeline rests on, so NL ambiguity would propagate everywhere) and instead inserts a small **constraint skeleton** with four slots — `trigger`, `role`, `gate`, `params` — so the LLM does only the fuzzy NL parsing and a deterministic step matches the skeleton to one of the templates above.

Algorithm 1 (verbatim shape)

```
Require: Skill specification S, reachability goals  $\Phi$ 
Ensure: Set of violations V
1:  $\kappa \leftarrow \text{seeds}(S, \Phi); V \leftarrow \emptyset$ 
2: repeat
3:    $t \leftarrow \text{choose}(\kappa), \omega \leftarrow \text{bandit}(\Omega)$ 
4:    $t' \leftarrow \text{mutate}(t, \omega)$ 
5:    $\tau \leftarrow \text{annotate}(\text{execute}(t'))$ 
6:   for each  $\phi \in \Phi$  do
7:     if  $\tau \models \phi$  then
8:        $V \leftarrow V \cup \{(t', \tau, \phi)\}$   $\triangleright$  violation found
9:    $s \leftarrow R(\tau, \Phi); \text{updateBandit}(\omega, s)$ 
10:  if  $s > \theta$  then  $\triangleright$  admit promising mutant
11:     $\kappa \leftarrow \kappa \cup \{t'\}$ 
12: until terminated
13: return V
```

Operator family Ω contains 16 LLM-driven mutators in four families: **Confirmation Weakening** (e.g. double-negative replies, delegated-consent claims), **Precondition Skipping** (urgency framing, authority claim), **Action Reframing** (paraphrasing the forbidden verb), **Resource Redirection** (domain-adjacency, numeric-edge values). A Thompson-Sampling bandit concentrates effort on the most productive operator–goal combinations; mutation prompts explicitly forbid imperative overrides, system-role impersonation, and known jailbreak patterns so the input set stays inside the **benign** boundary that defines specification violations (as distinct from prompt injection).

Headline numbers

On 402 real-world skills from the largest public agent-skill marketplace (OpenClaw / ClawHub), spanning six domains from crypto-finance to cloud infrastructure:

Metric	Result
Skills with at least one specification violation	120 / 402 (29.9 %)
Previously-unknown exploitable guardrail violations in deployed skills	26 zero-day
Average fuzzing convergence time	~11 min per skill
Ablation — remove semantic-mutation engine	–53 % of discovery
Ablation — remove Thompson-Sampling bandit	–35 % of discovery
Ablation — remove goal-proximity feedback	additional –29 %
Recurring specification pitfalls distilled	6

Why it matters

Three contributions sit on top of the empirical result. *First*, the formal model is the right level of abstraction for the next generation of agent-safety benchmarks — it makes the violation oracle deterministic and

reproducible, removing the LLM-as-judge dependency that contaminates most prior work in this area. *Second*, the constraint-skeleton interface is a reusable engineering pattern: confine NL ambiguity to one narrow, inspectable interface, then let everything downstream be mechanical. *Third*, the 29.9 % violation rate on *real deployed skills* with *active user bases* is the kind of empirical data point that will reshape how skill marketplaces gate publication. The six specification pitfalls in the paper read like a starter style guide for safer skill design.

1.4 · BINARY-VULN-AGENT · Veritas combines a static slicer over RetDec-lifted LLVM IR, a dual-view LLM detector that reasons over decompiled C with selective IR snippets, and a debugger-backed validator — 90 % recall on real-world binaries, 0 false positives across 623 exhaustively-verified candidates, and a new Apple CVE

1.4 Veritas: A Semantically Grounded Agentic Framework for Memory Corruption Vulnerability Detection in Binaries

[arXiv:2605.15097](https://arxiv.org/abs/2605.15097)

Title: Veritas: A Semantically Grounded Agentic Framework for Memory Corruption Vulnerability Detection in Binaries **Authors:** Xinran Zheng, Alfredo Pesoli, Marco Valleri, Suman Jana, Lorenzo Cavallaro **Affiliations:** University College London · Bynario · Columbia University **Venue:** arXiv preprint, 2026-05-14 (ACM template). **License:** ACM template default; figures not redistributable. **Source signal:** Scholar alert “Recommended articles” 2026-05-24; preference-promoted preprint.

Thesis

Detecting memory corruption vulnerabilities (MCVs) in stripped binaries is a problem of **semantic reconstruction**: compilation and stripping erase object boundaries, pointer relations, lifetimes, and high-level control flow, so the analysis must rebuild *how memory objects are allocated, propagated, constrained, and accessed across functions* from artefacts that no longer carry the information. Recent LLM-based agentic frameworks improve code reasoning but compensate for the lost semantics with summarization, retrieval, or call-graph expansion — none of which directly solve the recovery problem for stripped binaries, because call-graph expansion misses shared-state dependencies beyond direct caller–callee edges, summarization discards the precise facts a propagation-path assessment needs, and coverage-guided fuzzing explores too broadly to reach the specific path-and-memory-state required to trigger a given corruption within a bounded budget. The paper’s position: **the core issue is not model capability, but the absence of explicit static and executable grounding**.

Architecture

Veritas organises analysis into three stages aligned with two grounding layers (static and runtime):

1. **Slicer (defuse)** over RetDec-lifted LLVM IR. Reconstructs interprocedural value-flow relations from parser-derived LLVM-IR facts — def-use, call, return, global access, pointer-operation relations — and emits compact, witness-backed source-to-sink flow objects. The slicer’s job is to give the downstream LLM *only* the propagation evidence it needs, not the whole binary.
2. **Dual-view Detector (discovery)**. Performs step-wise reasoning over each flow object using **decompiled C as the primary view** and **selectively-instantiated LLVM IR at slicer-identified vulnerability-relevant loci**. This is the paper’s resolution to a precision/efficiency dilemma: low-level facts needed to preserve object identity and sink semantics are more explicit in lifted IR,

while higher-level structure for interpreting vulnerability logic is clearer in decompiled code, so neither representation alone suffices.

3. **Multi-agent Validator.** Translates each candidate vulnerability into targeted execution checks — guided debugging, breakpoint inspection, memory-checking oracles such as Valgrind — and confirms or refutes the hypothesis with debugger-visible artefacts and concrete runtime evidence. Sample output from the paper’s motivating CVE-2020-13790 walkthrough: *“Valgrind: Invalid write of size 8, 0 bytes after a block of size 0 alloc’d. SIGSEGV confirmed.”*

The three **design insights** (§2.2) read as portable principles for any LLM-in-the-loop program-analysis system: (1) ground context selection in memory-flow evidence; (2) selectively fuse dual representations to overcome the precision/efficiency dilemma; (3) ground feasibility confirmation in executable evidence.

Headline numbers

Metric	Result
Recall on curated benchmark of real-world binary memory-corruption cases	90 %
Exhaustively-validated detector candidates	623
False positives in the exhaustive subset	0
False positives identified by additional audit on larger cases	2 (manually confirmed)
Real-world case study	Previously-unknown Apple vulnerability discovered, confirmed, and assigned a CVE

Why it matters

Veritas is the only paper in this batch that closes the loop end-to-end on a stripped-binary target *and* lands a confirmed CVE on real third-party software. The dual-view detector is deployable engineering wisdom (decompiled C is what the LLM is good at; LLVM IR is what it needs at the few loci where decompilation lies); the validator’s reliance on debugger oracles rather than LLM-as-judge is the same principle that makes BMC-Agent (1.1) and Sefz (1.3) believable. Reusable for any agentic vulnerability pipeline that has to operate against compiled artefacts rather than source.

Keep — Secondary Reads

2.1 · SE-SOK · A SecDev ’26 systematisation of 225 symbolic-execution papers (15-year window) into five canonical modules (Execution Driver, Search Heuristic, Symbolic State Manager, Environment & Memory Model, Constraint Solver), plus a second SoK of 66 HCI/security papers yielding six usability guidelines — the modular vocabulary the rest of the agentic-SE lineage will get described in

2.1 SoK: A Modularized Framework for Symbolic Execution and Application for Usable Tool Design

[DOI 10.1145/3805773.3805996](https://doi.org/10.1145/3805773.3805996) · [Hosted copy](#)

Authors: James Mattei, Andrew Lin, Jasper Geer, Jie Hu, Moritz Schloegel, Tiffany Bao, Daniel Votipka
Affiliations: Tufts University · University of British Columbia · Arizona State University · CISA Helmholtz Center for Information Security **Venue:** ACM SecDev '26, Montreal — 5–6 July 2026. **License:** **Creative Commons Attribution 4.0 International** (CC-BY 4.0). **Source signal:** Scholar alert “Recommended articles” 2026-05-24.

The systematisation targets two research goals: (RG1) enable the continued advancement of symbolic execution for itself and its existing applications, and (RG2) facilitate SE’s expansion into new problem domains. The authors qualitative-coded 225 SE papers from the past 15 years across top cybersecurity and software-engineering venues, distilled five canonical modules — **Execution Driver, Search Heuristic, Symbolic State Manager, Environment and Memory Model, Constraint Solver** — and then layered a second qualitative review of 66 papers on developer & security-professional (DSP) tool usability from HCI/security/SE venues, yielding six usability guidelines. The two analyses are combined to identify, per module, the research needed to improve usability for future SE practitioners.

The paper’s framing of the four canonical SE bottlenecks (path explosion, costly constraint solving, path divergence, external function/environment modelling) and its mapping of dynamic symbolic execution (concolic) into the same modular vocabulary make it the standard “where does my contribution live?” reference for any future SE-applied paper — and a natural citation anchor for the Soteria/Sefz/BMC-Agent lineage in §1.1–§1.3 of this report.

2.2 · JAVA-WEB-INJECT · A TOSEM 2026 attack on the modern (Spring-era) Java web stack — entity-parameter-passing and configuration-driven injection paths that legacy servlet-era taint tooling does not catch

2.2 iDetector: Unraveling and Automating the Detection of Modern Java Web Injection Vulnerabilities

[DOI 10.1145/3808700](https://doi.org/10.1145/3808700)

Authors: Xiang Chen, Yu Li, Zhuo Jin, Yong Tan, Xiang Li, Bo Wang, Min Zhang et al. **Venue:** *ACM Transactions on Software Engineering and Methodology* (TOSEM), 2026. **License:** ACM. **Source signal:** Scholar alert “Recommended articles” 2026-05-24, position 0.

The thesis is that Java frameworks such as Spring simplify development but also open new injection-relevant control paths — entity-parameter passing, configuration-driven binding — that conventional taint analysis built around servlet-era source/sink patterns will not surface. iDetector targets those modern sinks specifically and reports an automated detection pipeline for them. Topical for any team running production Spring-class Java code; useful prior art for the next generation of framework-aware SAST.

2.3 · TRUSTWORTHY-ML-VULN · A TSE 2026 method for automated post-hoc identification of *untrustworthy* alerts emitted by ML vulnerability detectors that latch onto spurious code features

2.3 UNTRUSTVUL: Automated Untrustworthy Alert Identification in Vulnerability Detection Models

[IEEE Xplore 11522575](https://doi.org/10.1109/11522575)

Authors: Lam Nguyen Tung, Xiaoning Du, Neelofar Neelofar, Aldeida Aleti **Venue:** *IEEE Transactions on Software Engineering* (TSE), 2026. **License:** IEEE. **Source signal:** Scholar alert “Recommended articles” 2026-05-24, position 1.

ML vulnerability detectors have a well-documented tendency to predict using irrelevant code features (variable names, comment counts, formatting). UNTRUSTVUL automates the identification of detector alerts whose reasoning is spurious — i.e. alerts the operator should treat as untrustworthy even when the model is confident. Complements the agentic-validator lineage in §1: where Veritas and BMC-Agent push verification of an alert down to a deterministic backend, UNTRUSTVUL puts a trust filter in front of the alert itself.

2.4 · SATD-COEVOLUTION · A David-Lo-group empirical study showing 27 %/40 % of Dockerfile SATD admission/repayment events are coupled to non-Dockerfile artefacts — the methodological case for cross-file co-evolution as the unit of SATD analysis for IaC

2.4 Beyond the Tip of the Iceberg: Understanding SATD in Dockerfiles through the Lens of Co-evolution

[arXiv:2605.21238](https://arxiv.org/abs/2605.21238)

Authors: Wei Minn, Yan Naing Tun, Biniam Fesseha Demissie, Rui'ang Hu, Jiakun Liu, Mariano Ceccato, Lwin Khin Shar, David Lo **Venue:** arXiv preprint, 2026-05-20. **License:** arXiv non-exclusive distribution. **Source signal:** Scholar alert “David Lo - 新文章” 2026-05-24; preference-promoted preprint.

Prior SATD work treated Dockerfiles in single-file isolation. This paper widens the unit of analysis to include the production / test / build / configuration files a Dockerfile co-evolves with, and runs the numbers: approximately **27 % of SATD admission events** and **40 % of SATD repayment events** in Dockerfiles couple to non-Dockerfile artefacts; coupled SATD is repaid significantly faster overall ($p = 0.0201$), but coupled SATD that concerns missing functionalities persists longer than isolated counterparts; open and axial coding of the coupled events identifies external-dependency issues (especially unreleased upstream packages and bug fixes) as the most common admission trigger, and architectural refactoring as the most common repayment prerequisite. The methodological move — measure IaC-artefact debt with cross-file coupling rather than single-file slices — generalises to Terraform, Kubernetes manifests, GitHub Actions workflows, and any other IaC surface.

Borderline High-Priority

3.1 · ROBOTICS-DRL · Followed-researcher promotion only (Shang-Wei Lin co-author) — a Fuzzy Sets and Systems 2026 paper on fuzzy-logic-augmented DDPG for continuous mobile-robot motion planning; off-topic for program analysis / vulnerability detection

3.1 Fuzzy-DDPG: Integrating Fuzzy Logic with Continuous Deep Reinforcement Learning for Mobile Robot Motion Planning

[ScienceDirect S0165011426002046](https://www.sciencedirect.com/sci/article/pii/S0165011426002046)

Authors: Fubin Wu, Wei Tang, Yifan Zhou, Shang-Wei Lin, Hua Hu, Yang Liu, Zhongmin Ding **Venue:** *Fuzzy Sets and Systems*, 2026. **License:** Elsevier. **Source signal:** Scholar alert “Shang-Wei LIN - new articles” 2026-05-24 (followed-researcher promotion).

Combines fuzzy-logic action selection with the continuous action space of DDPG to address sharp-turn / limited-flexibility trajectories produced by discrete-action DRL on mobile robots. Carried by followed-researcher signal only; off-topic for the rest of this report's vulnerability- and program-analysis focus.

Cross-Paper Synthesis

A unifying methodological move runs through all four Outstanding deep-reads: **bound the LLM to semantic judgment, and discharge every soundness-relevant verdict on a deterministic backend.** BMC-Agent (1.1) names the principle outright ("agents propose, solvers verify") and instantiates it with CBMC and Kani. Veritas (1.4) does it with Valgrind and a debugger. Sefz (1.3) does it by reducing natural-language guardrails to a graph-reachability query whose oracle is a mechanical predicate-set membership check, explicitly refusing to let the LLM emit the goal directly. Teaching LLMs (1.2) does it at training time, treating Soteria symbolic-execution traces — produced by a sound engine — as the semantic anchor the model learns to reason over. The interface in each case is narrow enough to inspect, and that is the point: the value of the LLM is its semantic creativity inside a deterministic envelope, not its judgment about whether a bug is real.

A second pattern is **trace-as-first-class-citizen**. Sefz annotates traces with security predicates and reasons over labelled DAGs; Veritas emits witness-backed flow objects from a static slicer; BMC-Agent's four-stage validation pipeline is in effect a counterexample-trace classifier; Teaching LLMs trains on Soteria traces directly. The SoK paper (2.1) provides the modular vocabulary in which all four of these contributions can be re-described — Execution Driver, Search Heuristic, Symbolic State Manager, Environment & Memory Model, Constraint Solver — and is the citation that the next year of agentic-SE work will pivot on.

A third, smaller pattern across 1.4, 2.2, and 2.3 is the deliberate widening of the *negative* result. Veritas opens with a failure case showing a frontier coding agent on source code missing the bug that Veritas catches on the stripped binary. BMC-Agent has a "Contrast with LLM-only verification" subsection. UNTRUSTVUL exists because ML-vulnerability detectors have a known failure mode. Reviewers' default question is "couldn't a frontier model already do this?" — the convention is now to answer it explicitly in §1.

Writing & Rationale Insights

Three observations on how these papers are written, for our own future drafts:

State the slogan first, then operationalise it. BMC-Agent (1.1) puts *agents propose, solvers verify* on the first page; Sefz (1.3) introduces *Reachability Goals over Annotated Execution Traces* in the abstract. The methodological reader will not commit to a long pipeline diagram without the principle that makes it worth reading. This is the right way to write any paper whose contribution is architectural.

Negative ablations make the headline number believable. Teaching LLMs (1.2) reports its +17.9 pp result alongside the fact that trace pretraining alone is $\approx$ no gain and CoT alone is $\approx$ no gain — the **superadditivity** is the result, not the absolute number. Sefz's three ablations (−53 %, −35 %, −29 %) similarly justify each architectural choice. Frame ablations as load-bearing, not bookkeeping.

Confine NL ambiguity to one inspectable interface. Sefz's constraint skeleton is the cleanest example: an LLM does the fuzzy work of turning a guardrail into four slots, and a deterministic step then matches the skeleton to a goal template. The rationale ("the goal is the ground truth that the entire analysis is built on, so any ambiguity in NL understanding propagates as false positives or false negatives across every fuzzing iteration") is the canonical justification for IR-mediated LLM pipelines and will be quoted in this lineage going forward.